README.md

LLM Multi-Agent 2D Simulation

概要

LLMエージェントを複数用いて長期的な社会現象を観察するための汎用的な2Dマルチエージェントシミュレーション。ターンベースの世界観でターンごとに各エージェントへ状況を渡し、LLMの応答を解釈して行動を反映させ、その結果を次ターンへ引き継ぐ構造を採ります。このシステムの最小単位は世界観、ルール、エージェントの3要素で構成します。エージェントの数や性格、世界観の設定はすべてconfig.yaml側で定義します。

フェーズ分けや観測対象（どのタイミングで何を記録し何を比較するか）の判断は、各種実験設定ごとに個別に設定（hypothesis）します。シミュレーション実行方法はターンベースです。

```
python simulation.py
```

プロ野球、大会運営等は、単純な設定によりますが、自然言語・数値情報・地形情報・配列などの組み合わせ表現が可能なためLLMの理解・推論能力を活かす構造から扱えます。

File Event（大きなイベント）

シミュレーション中に発生する、通常の行動とは別の大きな出来事。エージェントの行動ログとは別ファイルに記録します。この種の出来事は、世界観・ルールの変化を伴うことが多いため、エージェントの行動ログとは別ファイルに記録します。

- 記録するもの（例）
  - 戦争・内乱（宣戦布告・停戦・和平合意・内乱発生・鎮圧・分裂など）
  - 大規模災害（地震・洪水・疫病・飢饉・大火・気象異常など）
  - 政権交代（リーダーシップ・体制・イデオロギーの変化）
  - 技術革新（発明・普及・廃棄）
  - 貿易・経済ショック（大規模な取引・関税・通貨・資源・市場の変化）
  - 宗教・文化・思想の変化（新宗教・改宗・禁教・文化流行・価値観の変化）
  - 外交関係（同盟・敵対・国交・条約・国境・領土の変化）
- 目的
  - 世界観・ルールの変化を伴うイベントを記録するため。
  - 長期的な観測・比較・考察のための基盤データとするため。

必要な環境

- Python 3.8以上
- Ollama（LLMサーバー）
- 必要なPythonパッケージ（requirements.txtを参照）
- Pillow（画像生成を行う場合のみ）

セットアップ

1. 依存関係のインストール

mac/Linuxの場合

```
# ターミナルでプロジェクトのあるフォルダへ移動
chmod +x setup_mac.sh
./setup_mac.sh

# 仮想環境を有効化
source venv/bin/activate
python -m spacy download ja_core_news_sm
pip install -r requirements.txt
```

Windowsの場合

```
REM コマンドプロンプトまたはPowerShellで実行
setup_win.bat

REM 仮想環境を有効化
venv\Scripts\activate.bat
venv\Scripts\activate.bat
python -m spacy download ja_core_news_sm
pip install -r requirements.txt
```

2. 依存関係の手動インストール

mac/Linuxの場合

```
source venv/bin/activate
```

Windowsの場合

```
venv\Scripts\activate.bat
```

3. Ollamaのセットアップ

Ollamaのインストール

- Ollamaをダウンロードしてインストールします。

Ollamaサーバーの起動

```
# ターミナルで常時起動
ollama serve
```

これにより、デフォルトで http://localhost:11434 でサーバーが起動します。

日本語・ハルシネーション対策（Qwen3系）

```
# 推奨モデルをダウンロード
ollama pull qwen3:14b
```

これにより、デフォルトで http://localhost:11434 でサーバーが起動します。

日本語・ハルシネーション対策（Qwen3系）

```
# 推奨モデルをダウンロード
ollama pull qwen3:14b
```

これにより、デフォルトで http://localhost:11434 でサーバーが起動します。

Ollamaサーバーの常時起動

```
# サーバーが起動しているか確認
curl http://localhost:11434/api/tags
```

正常に応答してJSONが返ってくれば、サーバーは起動しています。

モデルのダウンロードと選択

利用可能なモデルの確認

```
# インストールされているモデルの一覧を表示
ollama list
```

モデルのダウンロード

```
# 推奨モデルをダウンロード（例: llama3.2）
ollama pull llama3.2

# または別のモデルを指定する場合
ollama pull mistral
ollama pull qwen2.5
ollama pull gemma2:9b
```

モデルの選択

- シミュレーションは config.yaml の llm.model で指定されたモデルを使用します。
- 変更： config.yaml の llm.model を、ご利用中の環境にインストールされているモデル名に変更してください。

```
llm:
  model: "llama3.2"  # 使用するOllamaモデル名（ollama list で確認）
```

モデルの比較方法

- 推奨： ollama list で表示されたモデルを比較検討（例: qwen3:14b vs llama3.2）
- 確認項目： 日本語の自然さ、ハルシネーションの頻度、応答速度（特に重視する項目）
- 推奨モデル：
  - qwen3:14b ⭐⭐⭐ （推奨）
  - qwen3:8b ⭐⭐ （ollama list 確認）
  - llama3.2:3b-instruct ⭐⭐
  - llama3.2:8b ⭐⭐
  - qwen2.5:14b ⭐⭐ （推奨モデル不在時）

注意：

- 初回実行時は、指定されたモデルが自動でダウンロードされます。サイズが大きいモデルほど時間がかかります。
- config.yaml の llm.model で指定されたモデルが、ollama list で表示されるモデル名と完全一致していることを確認してください。
- 推奨モデルがダウンロードされていない場合、 config.yaml で指定されたモデルがダウンロードされ実行されます。
- ollama list で表示される name 列の値をそのまま指定するのが最も確実です。

4. 設定ファイルの確認

- config.yaml でパラメータを確認・調整
- 例： llm.model 等、ご自身の環境にインストールされているモデルに合わせて変更してください（ollama list で確認できます）

使用方法

注意：

- 実行前に依存関係を確認してください。
- Ollamaサーバーが起動していることを確認してください（ollama serve を別ターミナルで実行）

基本的な実行

```
# 仮想環境を有効化（毎回実行前に必要）
source venv/bin/activate  # mac/Linux
venv\Scripts\activate.bat  # Windows

# シミュレーションを実行
python main.py
```

可視化を有効にする

注意: tkinterがインストールされていない場合は、 --visualize オプションは無視されます。代わりに --save-frames を使用してください。

```
python main.py --visualize
```

フレームを保存する

```
python main.py --save-frames
```

カスタム設定ファイルを使用

```
python main.py --config custom_config.yaml
```

設定ファイル（config.yaml）

主要なパラメータ：

- simulation: シミュレーション設定
  - initial_total: シミュレーション開始時の総人口（例: 100）
  - half_population: 世界観の中央値（例: 50）。人口がこの値を超えると、世界観の成長率が半減します（ソフトキャップ）。
  - max_steps: シミュレーションの最大ターン数（例: 1000）
  - random_seed: 乱数シード（再現性のため）。null にすると毎回ランダムな結果になります。
  - message_history_limit: 保存する最大メッセージ数（デフォルト: 50）
  - conversation_turns: 1ターンあたりの会話ラウンド数（デフォルト: 1）
  - enable_reproduction: 繁殖を有効にするかどうか（true / false）
  - reproduction_cost: 繁殖に必要なエネルギー（デフォルト: 50）
  - initial_energy: 新規エージェントの初期エネルギー（デフォルト: 100）
  - max_age: 最大年齢（寿命）（デフォルト: 100）
  - base_mortality: 基礎死亡率（毎ターン死ぬ確率）（例: 0.01 = 1%）
  - overpopulation_penalty: 人口超過ペナルティ（例: 0.02）
  - resource_regen: 資源の自然回復量（例: 10）
  - resource_cost: 1回の行動で消費される資源（例: 1）
- llm: LLM（Ollamaなど）関連の設定
  - model: Ollamaで使うモデル名（例: qwen3:14b）
  - base_url: Ollamaのエンドポイント（例: http://localhost:11434）
  - temperature: サンプリング温度（低い値ほど一貫性が高い。高い値ほど多様な行動）
  - max_tokens: 応答の最大トークン数
  - request_timeout: 応答タイムアウト（秒）。ローカルLLMは遅いので長めに設定
  - request_limit: 全実行を通算した最大リクエスト数（null で無制限）。コスト管理に使う。
  - model_map: 特定のエージェントに別のモデルを割り当てる。キーにはエージェントのID、値にはモデル名。
- world: 世界観・ルール・エージェントの定義ファイルへのパス
- output: 結果の出力先
  - directory: 出力ファイルを保存するディレクトリ（例: "output"）
  - world_state: 世界の状態（人口・資源・イベント）を保存するファイル名（例: "world_state.jsonl"）
  - agents: エージェントの状態（位置・エネルギー・年齢・生存）を保存するファイル名（例: "agents.jsonl"）
  - conversations: 会話ログを保存するファイル名（例: "conversations.jsonl"）
  - summary: 最終サマリーを保存するファイル名（例: "summary.txt"）
  - save_every: 何ターンごとにファイルへ書き込むか（例: 1 = 毎ターン）
  - console_interval: コンソールに進捗を表示する間隔（例: 10 = 10ターンごとに表示）
- visualization: 可視化ツールの設定
  - enabled: 可視化を有効にするか（true / false）
  - port: ブラウザが接続するWebSocketポート（例: 8765）
  - grid_size: 1マスの表示ピクセル数（例: 10）
  - show_grid: グリッド線を表示するか（true / false）
  - show_resources: 資源の表示モード（"none" / "total" / "per_capita"）
  - show_labels: 人口ラベルを表示するか（true / false）
- world: 世界観・ルール・エージェントの定義ファイルへのパス
  - 例: world: "./world_definitions/medieval.yaml"
  - 相対パスはこのconfig.yamlからの相対パスとして解決されます。
- output: 結果の出力先
  - directory: 出力ファイルを保存するディレクトリ（例: "./output/medieval"）
  - world_state: 世界の状態（人口・資源・イベント）を保存するファイル名（例: "world_state.jsonl"）
  - agents: エージェントの状態（位置・エネルギー・年齢・生存）を保存するファイル名（例: "agents.jsonl"）
  - conversations: 会話ログを保存するファイル名（例: "conversations.jsonl"）
  - summary: 最終サマリーを保存するファイル名（例: "summary.txt"）
  - save_every: 何ターンごとにファイルへ書き込むか（例: 1 = 毎ターン）
  - console_interval: コンソールに進捗を表示する間隔（例: 10 = 10ターンごとに表示）
- visualization: 可視化ツールの設定
  - enabled: 可視化を有効にするか（true / false）
  - port: ブラウザが接続するWebSocketポート（例: 8765）
  - grid_size: 1マスの表示ピクセル数（例: 10）
  - show_grid: グリッド線を表示するか（true / false）
  - show_resources: 資源の表示モード（"none" / "total" / "per_capita"）
  - show_labels: 人口ラベルを表示するか（true / false）

出力

- output/ 配下に以下のファイルが生成されます。
  - world_state.jsonl: 世界の状態（人口・資源・イベント）。毎ターン1行（JSONL）。
  - agents.jsonl: エージェントの状態（位置・エネルギー・年齢・生存）。毎ターン1行（JSONL）。
  - conversations.jsonl: 会話ログ。毎ターン1行（JSONL）。
  - summary.txt: 最終サマリー。シミュレーション終了時に1回。
- 可視化ツールは world_state.jsonl と agents.jsonl を読みます。

開発ガイド

- ファイル形式: 全て JSONL（1行1JSONオブジェクト）。ストリーミング書き込みが可能。
- 時間: 全単位は「ターン」。1ターン = 1単位の時間。
- 人口ダイナミクス: 自然増加率 + 繁殖 - 基礎死亡率 - 過剰人口ペナルティ。ソフトキャップ方式。
- 資源: 人口 × resource_cost を消費し、resource_regen を回復。
- 繁殖: 資源が reproduction_cost を超えると確率で発生。

シミュレーション結果の可視化ツール

visualization/ ディレクトリに、シミュレーション結果を可視化するためのツールが含まれています。レポートを output/ ディレクトリに保存するだけで、ブラウザベースのインタラクティブな可視化ダッシュボードを起動できます。

フラクタビューア（viewer.html）

シミュレーションの時間ベクトルをフラクタル次元に変換し、視覚パターンのヒートマップ、分岐ビューへの遷移、生成ビューへの遷移を1つの画面で比較するための可視化ツールです。

使い方：

1. visualization/viewer.html をブラウザで開く
2. シミュレーション実行後に生成されたファイル（world_state.jsonl）をドロップ
   - Chrome / Edge 推奨。ブラウザによっては動作しないファイルがあります。
3. 「Analyze Files」ボタンを押す。ボタンが Analyzing... に変わり、処理が開始されます。
4. ステータスに「Analyzing N agents, M steps...」と表示されます。

操作方法：

操作	方法
時系列	▶ 再生、⏸ 停止、⏭ 最終 キー
表示ステップ	← 減少、→ 増加 または ↓ キー
ホットスポット	← 減少、→ 増加 または ↑ キー
ステップ間隔	スライダーをドラッグ
性能表示切替	右上のトグル（ローカル / 全体）

観測方法（generate_video.py）

simulation.py が保存したファイル（world_state.jsonl, agents.jsonl）を読み込んで、フレーム画像を生成し、ffmpeg でMP4動画にまとめます。

前提条件: Pillow と ffmpeg がインストールされていることを確認してください。Pillow は requirements.txt に含まれています。

```
# macOS
brew install ffmpeg

# Ubuntu/Debian
sudo apt install ffmpeg
```

使い方：

```
# 基本的な使い方（output/ ディレクトリから読み込む）
python visualization/generate_video.py output/

# ポイントのみ表示（エッジなし）
python visualization/generate_video.py output/ --no-edges

# ラベルを表示（ID と年齢）
python visualization/generate_video.py output/ --labels

# 出力先を指定（デフォルト: output/video.mp4）
python visualization/generate_video.py output/ -o video.mp4
```

オプション：

オプション	デフォルト	説明
input_dir（必須）	—	シミュレーションの結果ディレクトリ（world_state.jsonl と agents.jsonl が入っているもの）
-o, --output	input_dir/video.mp4	出力MP4ファイル名
--fps	10	フレームレート（1秒あたりのフレーム数）
--dpi	100	画像解像度（DPI）

LLMエージェント設計

基本設計

- 各エージェントは独立エージェント（Message / Memory / Action モジュールを持ち、同期的に行動）
- 同一のLLMコアを使用しペルソナのみ切り替えることで性格の違いを表現（世界観の違いは設定側で吸収）
- プロンプトは「観測の文字列」を受け取り、応答の文字列を返すだけの純粋な関数として扱います

ペルソナ（世界観）

- 各エージェントは固有の名前（name フィールド）を持ちます。これがログ・会話ログ・可視化で使われます。
- LLMプロンプトの冒頭で「You are [name]. You are a [role].」の形式で自己認識を与えます。
- 出力フォーマットの指定は世界観の設定ファイルに1行目から明示します。

Action（行動）

ルールの例（最小限）：

- 常に有効: 待機（何もしない）
- 隣接タイルが空: 移動
- エネルギー > 閾値: 繁殖
- 隣接エージェントが存在: 会話

Memory（記憶）

- LLMは持たない。memory フィールドのみ保存し、プロンプトに直近N件を含めます。
- 内部表現の管理は第2フェーズ以降。現時点では保存のみ。

- memory_limit: 保存する最大件数（例: 20）。古いものから破棄。
- memory_size: LLMに渡す直近件数（例: 5）。memory_limit 以下の値。

現在の実装状態

- 実装済み: Message（会話）のみ。Memory は直近N件を保存するだけ。Action は固定ルール（移動・繁殖・会話）。
- 未実装: 長期記憶の要約・検索、Action の LLM 選択、関係性・評判の管理。

シミュレーションステップの実行順序

各ターンでは以下の順序で実行されます：

1. 世界観のルールが1ターン進んだ状態で world_state を更新（人口・資源・イベント）。この値を元に後の処理が動きます。
2. Phase 1: エージェント応答 — 各エージェントが観測を受け取り、LLMを呼ぶかどうかを判断し、必要なら応答を生成します。
3. Phase 2: メッセージ配信 — 直前ステップで生成されたメッセージをエージェントへ渡します。
4. Phase 3: 行動処理 — 応答テキストを解釈し、移動・繁殖・会話（Message）を反映（世界観側のルールに従って）。
5. Phase 4: 保存処理 — ターン結果を jsonl ファイルに追記します。

この順序は固定です。ターンを跨ぐ処理の依存関係は、前ターンの結果のみを参照します。

LLM出力形式

メッセージ応答（Phase 3）：

```
"message": "広場のベンチに座っています。天気、良いですね。",
"conversation": true
```

行動選択（Phase 3）：

```
{
  "action": "move" or "stay",
  "direction": "up", "down", "left", or "right",
  "reasoning": "人が多い方へ移動してみよう。",
  "confidence": 0.8
}
```

ライセンス

GNU General Public License v3.0

詳細は LICENSE.txt を参照してください。